

Lab 8 - Uso Consciente de LLMs e Prompt Engineering

Mateus Espíndola

Junho 2026

1 Escolha da ferramenta

A LLM utilizada é o Claude Opus 4.8, com o plano de assinatura paga 'Max plan'. A ferramenta pode fazer buscas na Web e a data de uso é 11/06/2026.

2 Primeira tarefa: explicação conceitual

A fim de padronizar, todas subseções que forem relativas a um prompt serão identificadas com **[Prompt]** e, dentro delas, haverá a resposta gerada pela LLM. Comentários meus nessas subseções estarão em itálico e serão identificados pelo marcador *[NOTA DO ALUNO]*.

2.1 [Prompt]: Explique como a randomização funciona na Computação.

[NOTA DO ALUNO]: Logo de início, fui surpreendido tanto pela extensão da explicação da LLM, quanto pelo formato em que montou seu raciocínio. Desse modo, julguei válido solicitar que ela formatasse sua resposta para LaTeX, assim feito no próximo prompt.

2.2 [Prompt]: Adapte esse exato texto para Latex.

Na computação, “aleatoriedade” é um assunto mais sutil do que parece, porque computadores são máquinas **determinísticas**: dada uma mesma entrada, elas produzem sempre a mesma saída. Isso cria um paradoxo interessante — como uma máquina que sempre faz a mesma coisa pode gerar algo imprevisível? A resposta é que existem dois caminhos bem diferentes.

Pseudoaleatoriedade (o caso mais comum)

A maior parte da aleatoriedade que você encontra em programas não é aleatória de verdade. São os chamados **PRNGs** (geradores de números pseudoaleatórios). A ideia é começar com um valor inicial chamado **semente** (*seed*) e aplicar uma fórmula matemática repetidamente. Cada resultado vira a entrada do próximo cálculo, gerando uma sequência que *parece* caótica, mas é totalmente determinada desde o início.

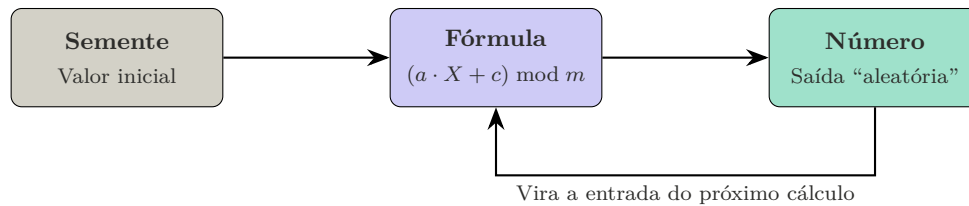
Um exemplo clássico e simples é o gerador linear congruente:

$$X_{n+1} = (a \cdot X_n + c) \bmod m$$

Escolhendo bons valores para a , c e m , os números resultantes ficam bem espalhados e “embaralhados”, passando em testes estatísticos de aleatoriedade — mesmo sendo 100% previsíveis se você souber a semente e a fórmula.

Uma consequência prática importante: **a mesma semente sempre produz a mesma sequência**. Isso parece um defeito, mas é uma vantagem enorme — permite reproduzir experimentos, depurar jogos, e refazer simulações idênticas. Por isso bibliotecas têm funções como `seed(42)`.

O ciclo abaixo ilustra esse funcionamento: a saída de cada passo “volta” para alimentar o próximo cálculo, o que torna o gerador determinístico.



Aleatoriedade verdadeira

Quando a previsibilidade é um problema sério — em criptografia, por exemplo — um PRNG comum não basta, porque um atacante que descobrir a semente prevê toda a sequência. Aí entram os geradores de números **verdadeiramente aleatórios** (TRNGs), que não usam fórmulas, e sim fenômenos físicos imprevisíveis como **ruído térmico** em circuitos, ruído atmosférico, decaimento radioativo, ou até a variação minúscula no tempo entre eventos de hardware (movimentos do mouse, jitter de teclado, atividade de disco).

Sistemas operacionais combinam essas fontes em um “reservatório de entropia”. No Linux, é daí que vêm os dispositivos `/dev/random` e `/dev/urandom`. Para segurança, usa-se uma categoria intermediária: os **CSPRNGs** (geradores pseudoaleatórios criptograficamente seguros), que começam com uma semente de entropia física verdadeira mas a expandem com algoritmos robustos o suficiente para que ninguém consiga prever os próximos valores nem voltar atrás.

Onde isso é usado

A escolha entre os tipos depende da finalidade. Para **simulações científicas** (método de Monte Carlo), jogos, embaralhar uma lista ou amostragem estatística, um PRNG rápido e reproduzível é o ideal. Para **senhas, chaves criptográficas, tokens de sessão e sorteios com dinheiro envolvido**, é obrigatório usar fontes seguras — usar `Math.random()` (um PRNG comum) para gerar uma chave de criptografia é uma falha de segurança clássica e perigosa.

Em resumo: a maior parte da “aleatoriedade” computacional é uma ilusão muito bem construída — sequências determinísticas que enganam todos os testes estatísticos. A aleatoriedade de verdade exige espiar o mundo físico, que é a única fonte realmente imprevisível que temos.

2.3 [Prompt]: Sou um estudante de primeiro semestre de Ciência da Computação. Elabore um texto explicando brevemente os conceitos-chave técnicos de como a randomização funciona em um sistema computacional. O formato do texto deverá ser em LaTeX e, levando em conta que o incluirei em meu próprio documento `.tex`, escreva-o tendo em mente que as seções de sua explicação serão incluídas dentro de subseções do meu próprio documento - ou seja, na prática, suas `\section's` serão `\subsubsections's`. Nesse sentido, também, não se preocupe com questões como título, packages e definições. Caso seja necessário o uso de alguma package ou definição específica para algum de seus diagramas, avise-me previamente, que incluirei em meu próprio documento de forma manual.

Computadores são máquinas **determinísticas**: para uma mesma entrada, produzem sempre a mesma saída. Isso gera uma tensão fundamental, já que gerar números *imprevisíveis* é justamente o oposto desse comportamento. Por isso, a randomização em computação é resolvida por duas estratégias distintas, descritas a seguir.

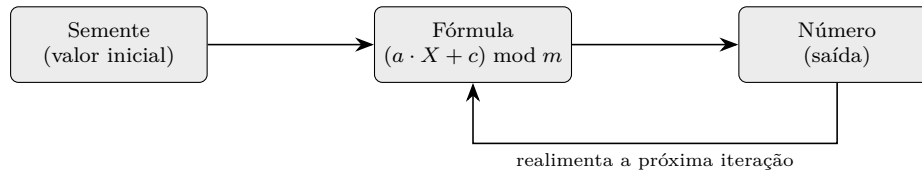
2.3.1 Geradores pseudoaleatórios (PRNGs)

Um **PRNG** (*Pseudo-Random Number Generator*) não produz aleatoriedade real: ele gera uma sequência que *parece* aleatória usando uma fórmula matemática determinística. Tudo parte de um valor inicial, a **semente**

(*seed*). A cada passo, a saída anterior é usada para calcular a próxima, formando um ciclo. Um exemplo clássico é o gerador linear congruente:

$$X_{n+1} = (a \cdot X_n + c) \bmod m$$

onde a , c e m são constantes bem escolhidas. A figura abaixo ilustra o ciclo: a saída de cada iteração realimenta a fórmula para produzir o próximo valor.



A principal característica de um PRNG é que **a mesma semente sempre gera a mesma sequência**. Longe de ser um defeito, isso garante *reprodutibilidade*: é o que permite repetir uma simulação ou depurar um jogo com resultados idênticos (daí funções como `seed(42)`). Em contrapartida, a sequência é totalmente previsível para quem conhece a semente e a fórmula.

2.3.2 Aleatoriedade verdadeira e entropia

Quando a previsibilidade é inaceitável, recorre-se a um **TRNG** (*True Random Number Generator*), que não usa fórmulas, mas mede fenômenos físicos genuinamente imprevisíveis, como ruído térmico em circuitos, ruído atmosférico ou a variação de tempo entre eventos de hardware. O sistema operacional acumula esses dados em um reservatório de **entropia**; no Linux, por exemplo, ele alimenta os dispositivos `/dev/random` e `/dev/urandom`.

2.3.3 Geradores criptograficamente seguros (CSPRNGs)

Entre os dois extremos estão os **CSPRNGs** (*Cryptographically Secure PRNGs*). Eles partem de uma semente de entropia física verdadeira, mas a expandem com algoritmos robustos, de modo que seja computacionalmente inviável prever os próximos valores ou reconstruir os anteriores. São obrigatórios para senhas, chaves criptográficas e tokens de sessão.

2.3.4 Qual usar em cada caso

A escolha depende do objetivo. Para simulações (método de Monte Carlo), jogos e amostragem estatística, um PRNG comum é rápido e tem a vantagem da reprodutibilidade. Já para qualquer aplicação de **segurança**, é indispensável usar um CSPRNG: empregar um PRNG comum como `Math.random()` para gerar uma chave é uma falha de segurança clássica, pois a saída se torna previsível.

2.4 Comparação entre as respostas

A semelhança entre as duas respostas é nítida, a mais provável causa disso é o fato de que os dois prompts foram feitos na mesma aba, de modo que a LLM reutilizou, no segundo, muitos conceitos do primeiro.

No entanto, nota-se, na segunda resposta, mais objetividade em suas explicações, aparentando ser um pouco menos didática. Além disso, nota-se mais qualidade na formatação do estilo LaTeX, tendo em vista que, na primeira resposta, é possível perceber a figura do diagrama deslocada em relação a onde deveria estar.

3 Segunda tarefa: Geração de código

3.1 [Prompt]: Escreva um programa que gera uma string aleatória.

```

"""
Gerador de strings aleatórias.

Demonstra as duas abordagens discutidas sobre randomização:
- random: PRNG comum, rápido e reprodutível (uso geral).
- secrets: CSPRNG, imprevisível (senhas, tokens, chaves).
"""

import random
import secrets
import string

def string_aleatoria(tamanho: int, alfabeto: str = string.ascii_letters + string.digits) -> str:
    """Gera uma string aleatória de uso geral usando o módulo 'random'.


Bom para testes, jogos e simulações. NÃO use para senhas: a saída  
é previsível para quem conhecer a semente do gerador.



:param tamanho: número de caracteres da string.  
:param alfabeto: conjunto de caracteres permitidos.  
:return: a string gerada.


    """
    return "".join(random.choices(alfabeto, k=tamanho))

def string_segura(tamanho: int, alfabeto: str = string.ascii_letters + string.digits) -> str:
    """Gera uma string aleatória CRIPTOGRAFICAMENTE SEGURA usando 'secrets'.


Use esta versão para senhas, tokens de sessão e qualquer coisa que  
precise ser imprevisível.



:param tamanho: número de caracteres da string.  
:param alfabeto: conjunto de caracteres permitidos.  
:return: a string gerada.


    """
    return "".join(secrets.choice(alfabeto) for _ in range(tamanho))

if __name__ == "__main__":
    TAMANHO = 12

    print("String de uso geral (random): ", string_aleatoria(TAMANHO))
    print("String segura (secrets):", string_segura(TAMANHO))

    # Exemplo de reprodutibilidade do PRNG: ao fixar a semente, a mesma
    # sequência é gerada em toda execução. Isso é útil para depuração.
    random.seed(42)
    print("Com semente fixa (42):", string_aleatoria(TAMANHO))

```

3.2 Análise e testes

Não especifiquei, nesse primeiro prompt, a linguagem de programação que a LLM deveria usar, a fim de descobrir a qual o agente naturalmente recorreria. Nesse sentido, a escolha ter sido Python não me surpreendeu.

Além disso, apesar da falta de profundidade em meu conhecimento sobre Python, não posso negar que a LLM documentou o código muito bem, de modo que facilitou-se muito minha compreensão do que cada função faz. Além disso, interessante o fato de terem sido incluídos no programa os dois métodos de randomização previamente explicados, tendo em vista que isso não consta no prompt.

3.2.1 Teste 1

```
String de uso geral (random):  gTP07x7hB81I
String segura      (secrets):  0f3t3UyOTSbd
Com semente fixa (42):        NbrnTP3fAbnF
```

3.2.2 Teste 2

```
String de uso geral (random):  hN50wQMWyz4J
String segura      (secrets):  xZDN1ldDgAPp
Com semente fixa (42):        NbrnTP3fAbnF
```

Como é de se notar, o programa tem êxito na geração de strings aleatórias, além de que o método PRNG gerou o mesmo resultado nos dois testes quando a seed foi a mesma.

4 Melhorando o Prompt

4.1 [Prompt]: Escreva um programa na linguagem C que, a partir de um inteiro entre 1 e 100, gera uma string aleatória. A string só pode conter letras e números. Não esqueça de incluir comentários.

```
#include <stdio.h>    /* printf, scanf          */
#include <stdlib.h>    /* rand, srand          */
#include <string.h>    /* strlen              */
#include <time.h>      /* time (semente do rand) */

int main(void)
{
    /* Alfabeto permitido: apenas letras (maiúsculas e minúsculas) e dígitos.
       As strings literais coladas lado a lado são concatenadas pelo compilador. */
    const char alfabeto[] = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
                             "abcdefghijklmnopqrstuvwxyz"
                             "0123456789";

    /* Quantidade de caracteres no alfabeto.
       strlen não conta o '\0' final, então este é o número exato de opções. */
    int total_caracteres = (int) strlen(alfabeto);

    int tamanho; /* tamanho da string desejado, informado pelo usuário */

    printf("Digite um inteiro entre 1 e 100: ");

    /* scanf retorna a quantidade de valores lidos com sucesso.
       Se não conseguir ler um inteiro, o resultado é diferente de 1. */
    if (scanf("%d", &tamanho) != 1) {
        printf("Entrada invalida: era esperado um numero inteiro.\n");
    }
```

```

        return 1; /* encerra com codigo de erro */
    }

    /* Validação: o valor precisa estar dentro do intervalo permitido. */
    if (tamanho < 1 || tamanho > 100) {
        printf("Erro: o numero deve estar entre 1 e 100.\n");
        return 1;
    }

    /* Inicializa o gerador pseudoaleatorio usando a hora atual como semente.
       Assim, cada execucao do programa gera uma sequencia diferente. */
    srand((unsigned int) time(NULL));

    /* Vetor que guarda a string. Como o tamanho maximo e 100, reservamos
       101 posicoes: 100 caracteres + 1 para o terminador nulo '\0'. */
    char resultado[101];

    /* Preenche cada posicao da string com um caractere sorteado do alfabeto. */
    for (int i = 0; i < tamanho; i++) {
        /* rand() % total_caracteres devolve um indice valido (0 ate total-1). */
        int indice = rand() % total_caracteres;
        resultado[i] = alfabeto[indice];
    }

    /* Toda string em C precisa terminar com o caractere nulo. */
    resultado[tamanho] = '\0';

    printf("String gerada: %s\n", resultado);

    return 0; /* sucesso */
}

```

4.2 Comparação

Apesar de o programa em C possuir muito mais limitações do que o script Python, ambos são resultados satisfatórios; o fato de serem programas extremamente triviais com certeza influenciou nisso.

Mesmo sem que isso seja solicitado, o primeiro programa mostrou-se didático e decentemente documentado. Enquanto isso, o segundo possui documentação ligeiramente mais completa (linha por linha), levando em conta que a inclusão de comentários foi solicitada no prompt.

5 Verificação Crítica

Analisarei o programa gerado em C.

5.1 A resposta estava correta?

Com base nos meus conhecimentos em C e nos testes feitos com o programa, não foi encontrado nenhum erro em seu código.

5.2 Havia informações incompletas ou incorretas?

Há uma pequena incompletude no fato de que a LLM, em sua documentação, não explicou o que ocorre caso um float seja colocado como input (o scanf lê apenas o número inteiro antes do '.').

No entanto, em geral, o programa mostra-se correto. Além disso, através desse programa, aprendi que a função `scanf()` retorna o número de elementos lidos.

5.3 Como a resposta foi verificada?

Com base nos meus conhecimentos em C e nos testes feitos com o programa.